

Chapter 2

Supervised Learning

Classification, regression,
and the major algorithm families

Learning Objectives

- Tell **classification** from **regression**
- Understand **generalization**, **overfitting**, and **underfitting**
- Know how each major algorithm works, and its **strengths/weaknesses**
- See how each model's **complexity** is controlled by its parameters
- Read model **uncertainty** estimates

This chapter doubles as a reference guide — come back when unsure about an algorithm.

Classification vs. Regression

Classification

- Predict a **class label** from a fixed list
- *Binary*: a yes/no question (spam / not spam)
- *Multiclass*: >2 classes (iris species, language of a web page)

Regression

- Predict a **continuous number** (float)
- Annual income, crop yield, house price

Quick test: is there *continuity* between outcomes? → regression.
(\$39,999 vs \$40,001 barely differ; English is not "between" French.)

Binary Classification Terms

- One class is called **positive**, the other **negative**
- "Positive" $\neq$ good — it's just the class you're looking for (e.g. *spam* = positive)
- Which class is positive is a domain choice

These terms matter later for **precision, recall, and ROC** (Chapter 5).

Generalization, Overfitting, Underfitting

We fit on **training data**, but we care about accuracy on **new, unseen** data — this is **generalization**.

Overfitting

- Model **too complex** for the data
- Memorizes training quirks
- Great on train, poor on test

Underfitting

- Model **too simple**
- Misses real structure
- Poor even on train

The Boat-Buying Example

A novice builds a rule: *"older than 45, fewer than 3 children or not divorced → buys a boat."*

- **100% accurate on the 12 training rows** — but hinges on tiny details
- A simpler rule (*"older than 50 → buys"*) is more trustworthy
- We want the **simplest model** that still predicts well

There's a sweet spot between too simple and too complex — that's the model we want.

Model Complexity vs. Dataset Size

- Complexity is tied to the **variety** in your data
- **More data → you can safely use a more complex model**
- Duplicating rows doesn't help; *more varied* data does

"Never underestimate the power of more data." Often collecting more beats tuning the algorithm.

The Book's Sample Datasets

Dataset	Type	Shape	Notes
forge	classification	26×2	synthetic, 2 classes
wave	regression	40×1	synthetic
cancer	classification	569×30	212 malignant / 357 benign
Boston	regression	506×13	extended to 104 w/ interactions

Small synthetic sets $\rightarrow$ *visualize* the ideas; real sets $\rightarrow$ *test at scale*.

Part 1

Classification & Regression Algorithms

k-Nearest Neighbors (k-NN)

Build the model = **store the training set**. Predict = look at the k closest points and **vote**.

```
from sklearn.neighbors import KNeighborsClassifier
clf = KNeighborsClassifier(n_neighbors=3).fit(X_train, y_train)
clf.score(X_test, y_test)      # forge: 0.86
```

- **Small k** → complex, jagged boundary (can overfit)
- **Large k** → smooth boundary, simpler model (can underfit)

k-NN for Regression

Predict the **average** target of the k nearest neighbors.

```
from sklearn.neighbors import KNeighborsRegressor
reg = KNeighborsRegressor(n_neighbors=3).fit(X_train, y_train)
reg.score(X_test, y_test)      # wave:  $R^2 = 0.83$ 
```

`score` returns **R^2** for regressors: 1 = perfect, 0 = predicts the mean.

k-NN: Strengths & Weaknesses

- ✓ Very **easy to understand**; good baseline with little tuning
- ✓ Two main parameters: **number of neighbors** and the **distance metric**
- ✗ **Slow** predictions on large datasets
- ✗ Poor on **many features**, and bad on **sparse** data (mostly zeros)

Easy to grasp, but rarely used in practice. The next family fixes both drawbacks.

Linear Models

Prediction is a **weighted sum of features**:

$$\hat{y} = w[0] \cdot x[0] + w[1] \cdot x[1] + \dots + w[p] \cdot x[p] + b$$

- **w** = coefficients/weights (`coef_`), **b** = intercept (`intercept_`)
- For 1 feature this is just a line; for many, a plane/hyperplane
- Models differ in **how** **w**, **b** **are learned** and **how complexity is controlled**

(Trailing `_` in scikit-learn marks values learned from data.)

Linear Regression (OLS)

Ordinary least squares — minimizes **mean squared error**. **No parameters → no way to control complexity.**

```
from sklearn.linear_model import LinearRegression
lr = LinearRegression().fit(X_train, y_train)
```

On the extended Boston data (104 features):

- Training $R^2 = \mathbf{0.95}$, Test $R^2 = \mathbf{0.61}$ → clear **overfitting**

We need a model that can control complexity.

Ridge Regression (L2)

Same formula, plus a constraint: keep coefficients **small** (close to 0). This is **regularization**.

```
from sklearn.linear_model import Ridge
Ridge(alpha=1.0).fit(X_train, y_train)    # Boston: 0.89 / 0.75
```

- **alpha** ↑ → simpler model, smaller coefficients (more regularization)
- **alpha=10** → 0.79 / 0.64 ; **alpha=0.1** → 0.93 / 0.77
- With **enough data**, regularization matters less (learning curves)

Lasso Regression (L1)

Also shrinks coefficients — but sets **some exactly to zero** → automatic **feature selection**.

```
from sklearn.linear_model import Lasso
Lasso(alpha=0.01, max_iter=100000).fit(X_train, y_train)
```

- Default `alpha=1` → underfits, uses only **4 of 105** features (0.29 / 0.21)
- `alpha=0.01` → **0.90 / 0.77** using 33 features — interpretable *and* accurate

Ridge vs Lasso: Ridge is the usual default; prefer Lasso when you expect few important features.

`ElasticNet` combines both.

Linear Models for Classification

Threshold the weighted sum at zero: $\hat{y} = w \cdot x + b > 0$.

The **decision boundary** is a line / plane.

```
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
```

- Two common algorithms: `LogisticRegression` and `LinearSVC`
- ⚠ Despite the name, `LogisticRegression` is a **classifier**

The Regularization Parameter C

For `LogisticRegression` / `LinearSVC`, `C` controls regularization — the *inverse* of Ridge's `alpha`.

- **High** `C` → fit training data hard (complex, less regularization)
- **Low** `C` → coefficients near zero, adjust to the "majority" (simpler)

On cancer data: `C=1` → 0.95 / 0.96 ; `C=100` → 0.97 / 0.97 ; `C=0.01` → 0.93 / 0.93

Use `penalty="l1"` for a sparse, interpretable model.

Linear Models: Strengths & Weaknesses

- ✓ **Fast** to train and predict; scale to very large datasets
- ✓ Strong on **high-dimensional** data (features $\geq$ samples)
- ✓ The prediction formula is easy to understand
- ✗ In **low dimensions**, other models may generalize better
- ✗ Coefficients can be hard to interpret when features are correlated

Naive Bayes Classifiers

Like linear models but even **faster to train** — learn simple per-class statistics for each feature independently.

Variant	Use for
GaussianNB	any continuous data
BernoulliNB	binary features (text)
MultinomialNB	count features (text)

- ✓ Blazing fast, great **baselines**, excellent on high-dim **sparse** text
- ✗ Generalization usually a bit **worse** than linear models; `alpha` smooths

Decision Trees

Learn a hierarchy of **if/else** questions (like 20 Questions).

- Tests on continuous data look like *"is feature $i > \text{value } a$?"*
- Splitting recursively until leaves are **pure** = 100% train accuracy → **overfit**

```
from sklearn.tree import DecisionTreeClassifier
DecisionTreeClassifier(random_state=0).fit(X_train, y_train)
# cancer: train 1.000 / test 0.937
```

Controlling Tree Complexity

scikit-learn does **pre-pruning** — stop the tree early.

```
DecisionTreeClassifier(max_depth=4, random_state=0).fit(X_train, y_train)
# cancer: train 0.988 / test 0.951 (better generalization)
```

Knobs: `max_depth`, `max_leaf_nodes`, `min_samples_leaf`.

Feature Importance

Trees report `feature_importances_` — how useful each feature was (0–1, summing to 1).

```
tree.feature_importances_      # e.g. "worst radius" dominates on cancer
```

- Always **non-negative**; unlike linear coefficients, they don't say *which* class
- Low importance $\neq$ useless — the feature may just be redundant

Decision Trees: Strengths & Weaknesses

- ✓ Easily **visualized** and explained to non-experts
- ✓ **No scaling needed**; handles mixed feature types
- ✗ Even with pruning, they tend to **overfit**
- ✗ Regression trees **cannot extrapolate** beyond the training range

Because single trees overfit, ensembles of trees are used in practice.

Part 2

Ensembles of Decision Trees

Random Forests

Many decision trees, each on a **bootstrap sample** and a random **subset of features** — then average. Averaging cancels individual overfitting.

```
from sklearn.ensemble import RandomForestClassifier
RandomForestClassifier(n_estimators=100, random_state=0).fit(X_train, y_train)
# cancer: test 0.972 (beats linear models & a single tree, no tuning)
```

- **n_estimators**: more trees = more robust (bigger/slower)
- **max_features**: lower → more diverse trees; `sqrt(n_features)` is a good default
- Set **n_jobs=-1** to use all CPU cores

Random Forests: Strengths & Weaknesses

- ✓ Among the **most widely used** methods; powerful with little tuning
- ✓ No scaling needed; **parallelizable**; more reliable importances than one tree
- ✗ Hard to **interpret** (hundreds of deep trees)
- ✗ Poor on **high-dimensional sparse** data (e.g. text) — use linear models there
- ✗ More **memory** and slower than linear models

Gradient Boosted Trees

Build **shallow** trees **serially**, each correcting the previous one's mistakes.

```
from sklearn.ensemble import GradientBoostingClassifier
GradientBoostingClassifier(random_state=0).fit(X_train, y_train)
# cancer: 1.000 / 0.958 (overfitting) → tune to fix
```

- Defaults: 100 trees, `max_depth=3`, `learning_rate=0.1`
- `learning_rate` ↓ or `max_depth` ↓ reduces overfitting
- `max_depth=1` → 0.991 / **0.972**

Gradient Boosting vs. Random Forests

- Often **slightly more accurate** than random forests, but **needs careful tuning**
 - Slower to train, faster to predict, smaller in memory
 - **Common workflow:** try a random forest first; move to gradient boosting to squeeze out extra accuracy
 - For large problems, consider the `xgboost` package
- ✗ Like all tree models: weak on high-dimensional sparse data

Kernelized Support Vector Machines

Add **nonlinear features** so a linear boundary becomes curved — done efficiently via the **kernel trick** (no explicit expansion).

- **RBF (Gaussian) kernel** is the common choice (`SVC`)
- Only the boundary-defining points matter — the **support vectors**

```
from sklearn.svm import SVC
SVC(kernel='rbf', C=10, gamma=0.1).fit(X_train, y_train)
```

Tuning SVM: C and gamma

- `gamma` — width of the Gaussian kernel: high $\rightarrow$ complex boundary; low $\rightarrow$ smooth
- `c` — regularization: high $\rightarrow$ fit points hard; low $\rightarrow$ simpler
- The two are **strongly coupled** — tune them together

⚠ **SVMs need scaling.** On unscaled cancer data: **1.00 / 0.63** (severe overfit).

After scaling to [0,1]: **0.95 / 0.95**; with `c=1000`: **0.99 / 0.97**.

Kernel SVM: Strengths & Weaknesses

- ✓ Powerful on **medium-sized** datasets; works in low *and* high dimensions
- ✓ Great when all features are on **similar scales / units**
- ✗ Scales poorly beyond **~100,000 samples** (runtime & memory)
- ✗ Needs **careful preprocessing** and parameter tuning
- ✗ Hard to **inspect** / explain

Neural Networks (MLPs)

Multilayer perceptrons stack weighted sums with **hidden layers** and a **nonlinearity** (relu or tanh) between them.

```
from sklearn.neural_network import MLPClassifier
MLPClassifier(hidden_layer_sizes=[100, 100], alpha=1,
              random_state=0).fit(X_train_scaled, y_train)
# cancer (scaled): 0.988 / 0.972
```

- Model complexity $\approx$ number of layers $\times$ units per layer
- **alpha** adds regularization (like Ridge / **C**)

Neural Networks: Strengths & Weaknesses

- ✓ Can build **very complex** models given enough data & compute
- ✓ Often beat other methods on large, **homogeneous** data
- ✗ **Long** training times; **sensitive to scaling** (scale to mean 0, variance 1) and to parameters
- ✗ Harder to tune; less effective on heterogeneous features

Strategy: build a network big enough to overfit, then add regularization (α).

For serious deep learning, use `keras` / `tensorflow` / `pytorch` (GPU support).

Part 3

Uncertainty Estimates

Why Uncertainty Matters

Beyond *which* class, classifiers can say *how confident* they are.

- Crucial when **error costs are asymmetric** (e.g. a false negative in cancer testing)
- Two methods: `decision_function` and `predict_proba`
- Most classifiers provide at least one; many provide both

The Decision Function

Returns one score per sample (binary case), shape `(n_samples,)`.

```
gbrt.decision_function(X_test)      # e.g. [4.14, -1.68, -3.95, ...]  
gbrt.decision_function(X_test) > 0  # recover the predicted class
```

- **Positive** → the "positive" class; **negative** → the other
- The scale is **arbitrary**, so raw values are hard to interpret

Predicting Probabilities

`predict_proba` returns a probability per class, shape `(n_samples, 2)`, summing to 1.

```
gbdt.predict_proba(X_test) # [[0.016, 0.984], [0.843, 0.157], ...]
```

- Easier to interpret than `decision_function`
- A model is **calibrated** if a "70% certain" prediction is right 70% of the time
- **! Overfit models look overconfident** — high probability $\neq$ correct

Uncertainty in Multiclass

Both methods extend to multiclass:

- **Shape becomes** `(n_samples, n_classes)` — one score/probability per class
- Recover the prediction with `argmax` **across columns**
- Map indices back to labels via `classes_` (important when labels are strings)

```
np.argmax(gbrt.predict_proba(X_test), axis=1)
```

When to Use Each Model

Model	Use when...
Nearest neighbors	small dataset, easy baseline
Linear models	first thing to try; large / high-dim data
Naive Bayes	classification only; even faster than linear
Decision trees	need a visual, explainable model
Random forests	robust, powerful default for most tabular data
Gradient boosting	top accuracy, willing to tune
SVMs	medium data, features on similar scales
Neural networks	large data, complex patterns, can tune & scale

Start simple → go complex only if needed.

Chapter Summary

- Supervised = **classification** (labels) or **regression** (numbers)
- Manage the **complexity** ↔ **generalization** trade-off; more data allows more complexity
- Each family trades off **speed, accuracy, interpretability, and scaling needs**
- **Scale features** for k-NN, SVM, and neural nets; trees/forests don't need it
- Tree **ensembles** are strong general-purpose defaults
- Use **uncertainty estimates** when the cost of errors differs

Next: Chapter 3 — Unsupervised Learning & Preprocessing